

TP

Déchiffrement AES

Le fichier **aes.py** est une implémentation en Python de l'AES.

PARTIE 1 & 2

Dans ce fichier, la fonction **S** (ligne 124) contient le code permettant de calculer la valeur **S(x)** comme explicitée en cours. Cette fonction est utilisée pour modifier chaque case de l'état courant (tableau à 4 lignes et 4 colonnes) par la transformation **SubBytes**.

Une case contenant un entier entre 0 et 255, on pourrait aussi précalculer toutes les images possibles de la fonction **S** en stockant dans une table les valeurs **S(x)** pour x variant de 0 à 255.

Utilisez la fonction **S** du script **aes.py** pour créer **automatiquement** la liste **SBOX** contenant toutes ces valeurs. Complétez le script afin d'afficher cette liste. Vous devez obtenir le résultat ci-dessous.

```
SBOX = [99, 124, 119, 123, 242, 107, 111, 197, 48, 1, 103, 43, 254, 215, 171, 118, 202, 130, 201, 125, 250, 89, 71, 240, 173, 212, 162, 175, 156, 164, 114, 192, 183, 253, 147, 38, 54, 63, 247, 204, 52, 165, 229, 241, 113, 216, 49, 21, 4, 199, 35, 195, 24, 150, 5, 154, 7, 18, 128, 226, 235, 39, 178, 117, 9, 131, 44, 26, 27, 110, 90, 160, 82, 59, 214, 179, 41, 227, 47, 132, 83, 209, 0, 237, 32, 252, 177, 91, 106, 203, 190, 57, 74, 76, 88, 207, 208, 239, 170, 251, 67, 77, 51, 133, 69, 249, 2, 127, 80, 60, 159, 168, 81, 163, 64, 143, 146, 157, 56, 245, 188, 182, 218, 33, 16, 255, 243, 210, 205, 12, 19, 236, 95, 151, 68, 23, 196, 167, 126, 61, 100, 93, 25, 115, 96, 129, 79, 220, 34, 42, 144, 136, 70, 238, 184, 20, 222, 94, 11, 219, 224, 50, 58, 10, 73, 6, 36, 92, 194, 211, 172, 98, 145, 149, 228, 121, 231, 200, 55, 109, 141, 213, 78, 169, 108, 86, 244, 234, 101, 122, 174, 8, 186, 120, 37, 46, 28, 166, 180, 198, 232, 221, 116, 31, 75, 189, 139, 138, 112, 62, 181, 102, 72, 3, 246, 14, 97, 53, 87, 185, 134, 193, 29, 158, 225, 248, 152, 17, 105, 217, 142, 148, 155, 30, 135, 233, 206, 85, 40, 223, 140, 161, 137, 13, 191, 230, 66, 104, 65, 153, 45, 15, 176, 84, 187, 22]
```

Modifier votre programme afin qu'il construise aussi **automatiquement** la liste **InvSBOX** vérifiant $\text{InvSBOX}[\text{S}(i)] = i$ et qu'il affiche son contenu. Vous devez obtenir le résultat ci-dessous.

```
InvSBOX = [82, 9, 106, 213, 48, 54, 165, 56, 191, 64, 163, 158, 129, 243, 215, 251, 124, 227, 57, 130, 155, 47, 255, 135, 52, 142, 67, 68, 196, 222, 233, 203, 84, 123, 148, 50, 166, 194, 35, 61, 238, 76, 149, 11, 66, 250, 195, 78, 8, 46, 161, 102, 40, 217, 36, 178, 118, 91, 162, 73, 109, 139, 209, 37, 114, 248, 246, 100, 134, 104, 152, 22, 212, 164, 92, 204, 93, 101, 182, 146, 108, 112, 72, 80, 253, 237, 185, 218, 94, 21, 70, 87, 167, 141, 157, 132, 144, 216, 171, 0, 140, 188, 211, 10, 247, 228, 88, 5, 184, 179, 69, 6, 208, 44, 30, 143, 202, 63, 15, 2, 193, 175, 189, 3, 1, 19, 138, 107, 58, 145, 17, 65, 79, 103, 220, 234, 151, 242, 207, 206, 240, 180, 230, 115, 150, 172, 116, 34, 231, 173, 53, 133, 226, 249, 55, 232, 28, 117, 223, 110, 71, 241, 26, 113, 29, 41, 197, 137, 111, 183, 98, 14, 170, 24, 190, 27, 252, 86, 62, 75, 198, 210, 121, 32, 154, 219, 192, 254, 120, 205, 90, 244, 31, 221, 168, 51, 136, 7, 199, 49, 177, 18, 16, 89, 39, 128, 236, 95, 96, 81, 127, 169, 25, 181, 74, 13, 45, 229, 122, 159, 147, 201, 156, 239, 160, 224, 59, 77, 174, 42, 245, 176, 200, 235, 187, 60, 131, 83, 153, 97, 23, 43, 4, 126, 186, 119, 214, 38, 225, 105, 20, 99, 85, 33, 12, 125]
```

Déposez sur Moodle votre script qui contient la version modifiée de **aes.py** permettant de créer les listes **SBOX** et **InvSBOX**, nommez votre script **aes-partie1.py**.

Modifiez la fonction **SubBytes** (ligne 143) du fichier **aes-partie1.py** afin qu'elle utilise la liste **SBOX** et qu'elle n'utilise plus la fonction **S**. Vérifiez que, suite à cette modification, votre programme renvoie les mêmes résultats que le programme original **aes.py**. Déposez sur moodle

votre script en le nommant **aes-partie2.py**.

PARTIE 3

En vous inspirant du script **aes-partie2.py** que vous venez de modifier, développez le script **aes-dechiffre.py** qui réalise le déchiffrement AES en suivant la même structure que le chiffrement AES, c'est-à-dire que la suite des opérations effectuées sont de même nature et sont exécutées **dans le même ordre** que celles effectuées lors du chiffrement. Votre script utilisera les listes **SBOX** et **INVSBOX**. A titre d'exemple, voici le déroulement du chiffrement et du déchiffrement de l'AES à 3 tours. On rappelle que la transformation **MixColumn** correspond à une opération matricielle. Cette matrice est notée **MC** dans le code ci-dessous, la matrice **MC⁻¹** est l'inverse de la matrice **MC**.

Voici la représentation Python de cette matrice inverse :

```
matrix_invmix_columns = [[0xe,0xb,0xd,0x9],[0x9,0xe,0xb,0xd],[0xd,0x9,0xe,0xb],  
[0xb,0xd,0x9,0xe]]
```

Chiffrement AES sur 3 tours

$$E \leftarrow E \oplus K_0$$

$$E \leftarrow BS(E)$$

$$E \leftarrow SR(E)$$

$$E \leftarrow MC \times E$$

$$E \leftarrow E \oplus K_1$$

$$E \leftarrow BS(E)$$

$$E \leftarrow SR(E)$$

$$E \leftarrow MC \times E$$

$$E \leftarrow E \oplus K_2$$

$$E \leftarrow BS(E)$$

$$E \leftarrow SR(E)$$

$$E \leftarrow E \oplus K_3$$

Déchiffrement AES sur 3 tours

$$E \leftarrow E \oplus K_3$$

$$E \leftarrow InvBS(E)$$

$$E \leftarrow InvSR(E)$$

$$E \leftarrow MC^{-1} \times E$$

$$E \leftarrow E \oplus (MC^{-1} \times K_2)$$

$$E \leftarrow InvBS(E)$$

$$E \leftarrow InvSR(E)$$

$$E \leftarrow MC^{-1} \times E$$

$$E \leftarrow E \oplus (MC^{-1} \times K_1)$$

$$E \leftarrow InvBS(E)$$

$$E \leftarrow InvSR(E)$$

$$E \leftarrow E \oplus K_0$$

A partir du cryptogramme :

```
['e4', '2f', 'ea', '68']  
['fe', '2a', '32', '25']  
['3f', '43', '89', '0e']  
['0d', 'c1', '5e', '2b']
```

renvoyé par le script **aes.py**, votre script doit afficher comme valeur de sortie le tableau suivant (utilisez la fonction **affiche**) :

['55', '65', '67', '6c']
['6e', '73', '65', '61']
['20', '73', '20', '69']
['6d', '61', '63', '72']

qui correspond au texte clair du script **aes.py**.

PARTIE 4

Modifiez le code du fichier **aes.py** de façon à pouvoir chiffrer des messages de plus de 16 caractères. Pour ne pas avoir à gérer des problèmes de padding, vous ne considérerez que le cas de messages dont le nombre de caractères est un multiple de 16. Pour réaliser le chiffrement vous utiliserez le mode CBC.

Par exemple, pour le message

J'adore vraiment la cryptographie, bien plus que le développement

(ATTENTION , il n'y a pas d'espace après la virgule situé entre les mots cryptographie et bien)

qui correspond aux 4 états suivants :

['4a', '6f', '76', '6d']	['20', '63', '74', '61']	['65', '65', '6c', '71']	['6c', '65', '6f', '6d']
['27', '72', '72', '65']	['6c', '72', '6f', '70']	['2c', '6e', '75', '75']	['65', '76', '70', '65']
['61', '65', '61', '6e']	['61', '79', '67', '68']	['62', '20', '73', '65']	['20', '65', '70', '6e']
['64', '20', '69', '74']	['20', '70', '72', '69']	['69', '70', '20', '20']	['64', '6c', '65', '74']

avec IV=1234567890abcdef1234567890abcdef et la clé K du script **aes.py**, vous devez obtenir le cryptogramme final :

['22', '0c', 'd5', '9b']	['cb', '66', 'bc', '6b']	['be', '79', '42', '17']	['a1', 'f1', 'de', '0b']
['a0', '23', '42', 'a2']	['72', 'b9', '34', '75']	['e1', 'a2', 'dd', 'e6']	['b4', '23', '77', '81']
['b6', 'dd', '87', '13']	['4b', '29', '97', '0f']	['f7', 'c3', '6a', '96']	['e0', '4f', '30', 'a0']
['e2', '5a', '42', 'd3']	['43', '99', '22', '85']	['4e', 'f7', '5b', 'd1']	['12', '6b', '84', '7a']

Votre script devra s'appeler **aes-cbc.py**.